

TP LEGO Segway

Placement de pôles (PID numérique vs retour d'état)

La durée de ce TP est de 8 heures.

Ce TP est construit sur la base du code du Lego Segway proposé par Yoriyama Yamamoto [<http://www.mathworks.com/matlabcentral/fileexchange/19147>].

Ce sujet de TP est sous license CC-by-SA : 

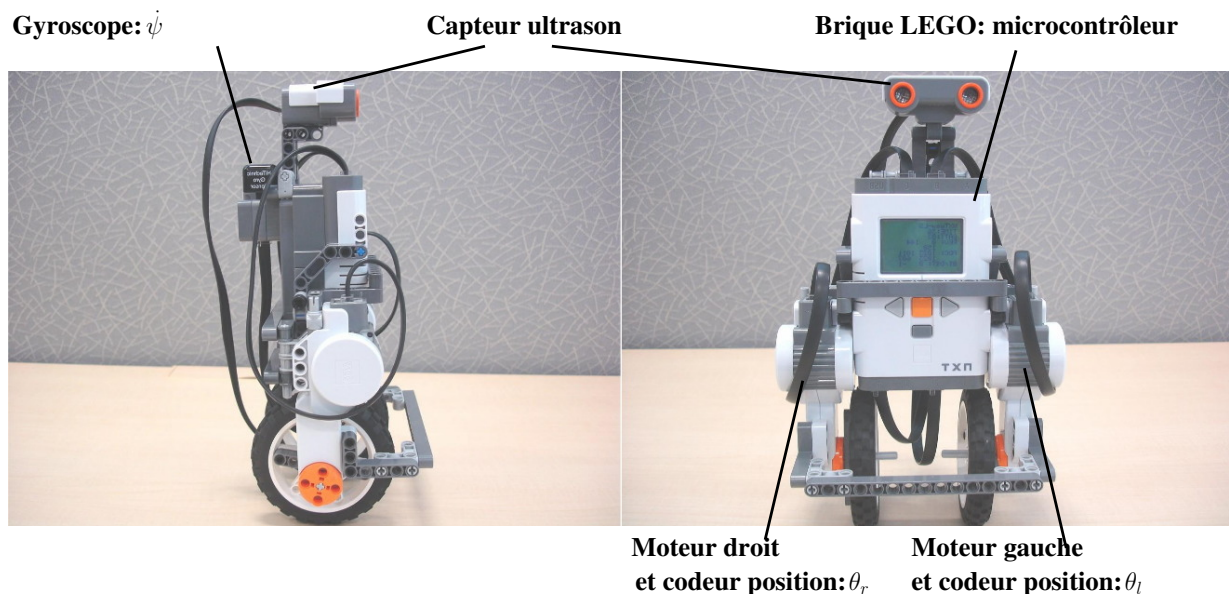


FIGURE 1 – Photo du lego Segway étudié

1 Introduction au TP

1.1 Objectif

L'objectif est de maintenir à l'équilibre le LEGO Segway (figure 1), qui est assimilable à un pendule inversé. Ce système a la particularité d'être naturellement instable en boucle ouverte.

On définit ψ , l'angle entre le corps du Segway et la verticale (figure 2); et θ , l'angle de rotation des roues par rapport au sol tel que la position au sol du Lego soit $x = R\theta$. L'asservissement consiste alors à commander U , la tension commune aux bornes des 2 moteurs afin de :

- maintenir le corps du Segway à l'équilibre (réguler à 0 l'angle ψ),
- asservir θ (ou $\dot{\theta}$) pour asservir la position au sol du Segway dans la direction $\vec{x}$.

On peut utiliser dans ce but :

- des correcteurs PID numériques avec une synthèse directe dans le domaine échantillonné, ou une synthèse à temps continu avant d'obtenir la version échantillonnée/à temps discret via la transformée bilinéaire.
- ou un retour d'état (cf. cours 1A) calculé sur le modèle échantillonné du système.

Dans ce TP, on utilise le logiciel Matlab pour la synthèse et Simulink pour la simulation des correcteurs et la génération du code C qui sera implanté dans la brique Lego.

1.2 Descriptif des entrées et sorties du système

L'entrée du système est la tension d'induit U commune aux 2 moteurs ($U = U_r = U_l$). Dans un premier temps, notre segway n'est pas prévu pour savoir tourner.

Entrée	Commande	Résolution	Plage
Tension d'induit du moteur U (Volt)	Vitesse du rotor	1% de la tension de la batterie (≈ 8 Volts à pleine charge)	$-100\% < < +100\%$

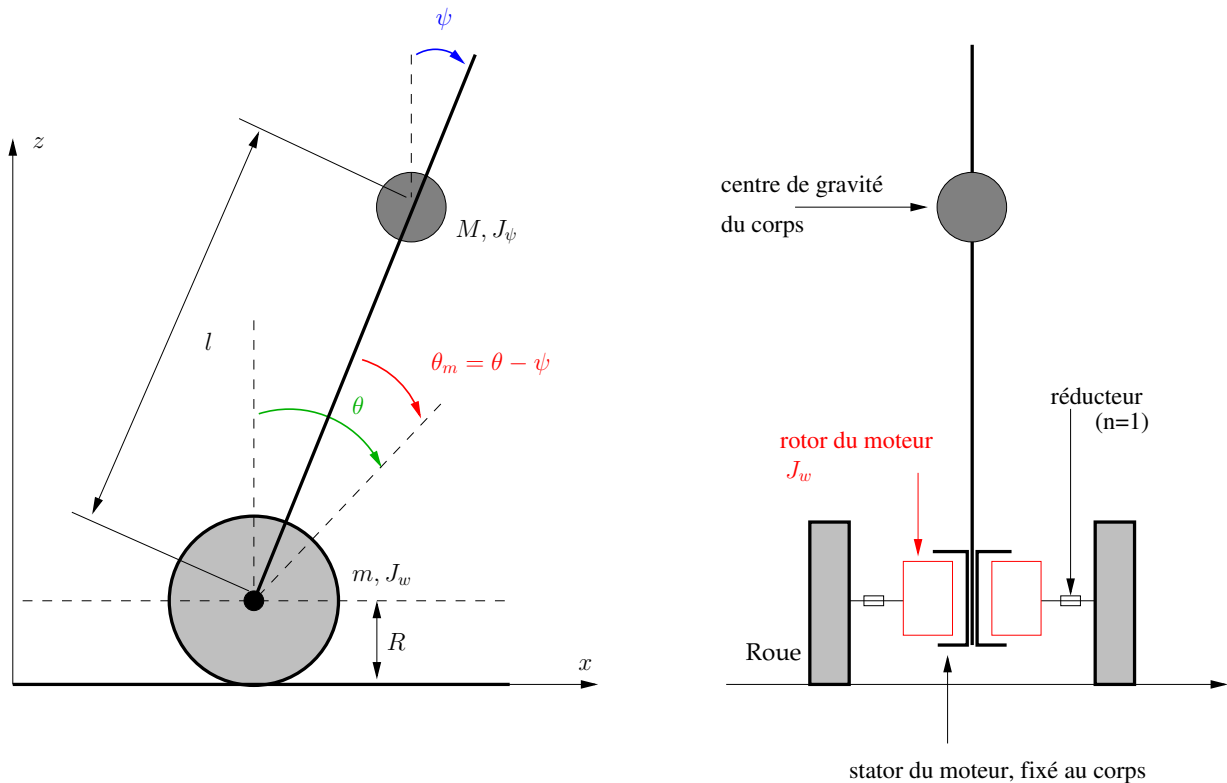


FIGURE 2 – Paramétrisation du Lego Segway

Capteur	Mesure	Résolution
Codeur incrémental moteur	$\theta_{m\{l,r\}}$: angle entre roues {right/left} et corps	1 deg
Gyroscope	$\dot{\psi}$: vitesse angulaire du corps	1 deg/s, hors biais

Les 2 grandeurs ψ , angle entre le corps du robot et la vertical, et θ , angle moyen des roues par rapport au sol ne sont pas directement accessibles à la mesure via les capteurs disponibles. On peut cependant estimer la valeur de ces 2 grandeurs ($\hat{\psi}$ et $\hat{\theta}$) par les procédés suivants :

- intégration au cours du temps de $\dot{\psi}$ et suppression de l'offset par un filtrage passe-haut¹.
- $\hat{\theta} = \theta_m + \hat{\psi}$, avec $\theta_m = \frac{\theta_{mr} + \theta_{ml}}{2}$: angle moyen entre les roues et le corps, donné par les codeurs contenus dans les moteurs.

2 Préparation : Modèle dynamique du système

Les équations d'Euler-Lagrange appliquées au Segway donnent les équations dynamiques non linéaires suivantes pour son mouvement, en fonction des couples moteurs :

1. L'intégration sans précautions d'une vitesse n'est pas une bonne idée, car tout capteur analogique a un biais de mesure non nul (même si infiniment petit) qui a pour conséquence une divergence plus ou moins rapide de la mesure : $\int_0^t (\dot{x}(t) + \text{petit.offset}) dt = x(t) + \text{petit.offset} \times t$.

$$[(2m + M)R^2 + 2(J_w + J_m)] \ddot{\theta} + [MRl \cos(\psi) - 2J_m] \ddot{\psi} - MRl \sin(\psi) \dot{\psi}^2 = C_\theta \quad (1)$$

$$[Ml^2 + J_\psi + 2J_m] \ddot{\psi} + [MRl \cos(\psi) - 2J_m] \ddot{\theta} - Mgl \sin(\psi) = C_\psi \quad (2)$$

avec

l : distance axe de la roue-centre de gravité du corps

m : masse de la roue

M : masse du corps (brique)

J_w : moment d'inertie d'une roue

J_ψ : moment d'inertie du corps

J_m : moment d'inertie du rotor d'un moteur

R : rayon de la roue

g : accélération de la gravité

C_θ : Couples appliqués aux roues

C_ψ : Couple appliqué au corps

Note : Ce modèle physique n'est valide que si les angles sont exprimés en radians et tension en volt.

2.1 Modèle linéaire du système en fonction de la tension moteur

Le calcul d'une commande pour stabiliser le Segway nécessite un modèle linéaire (le cours n'a abordé que ce type de système) et ayant pour entrée la tension moteur U .

1. Linéariser le système d'équations (1) et (2) autour de la position d'équilibre du système ($\psi = 0$) en fonction des grandeurs ψ , θ et leurs dérivées : utiliser un développement limité à l'ordre 1 des fonctions trigonométriques autour de 0 et négliger les derniers termes non linéaires.
2. On néglige le frottement des roues sur le sol (hypothèse de mouvement sans glissement). Les 2 moteurs sont situés entre les roues et le corps, on a alors l'égalité suivante ("principe d'action-réaction") :

$$C_\theta = -C_\psi \quad (3)$$

et

$$C_\theta = 2.(C_m - f_m(\dot{\theta} - \dot{\psi})) \quad (4)$$

$$= 2.(k_t i_m - f_m(\dot{\theta} - \dot{\psi})) \quad (5)$$

avec

— C_m : le couple d'un moteur

— f_m : un coefficient de friction proportionnel à la vitesse de rotation du rotor

— k_t : la constante de couple du moteur

— i_m : l'intensité du courant dans l'induit du moteur

D'après le modèle du moteur à courant continu (voir section 2.1 du cours 1A), exprimer i_m en fonction de :

— k_b , la constante de la force contre-électromotrice

— U , la tension aux bornes de l'induit

— R_m , la résistance de l'induit (on néglige l'inductance du moteur)

Montrer alors que les équations (1) et (2), après leur linéarisation et l'expression des efforts en fonction de la tension d'induit U , peuvent s'écrire sous la forme :

$$E_{11} \ddot{\theta} + E_{12} \ddot{\psi} + 2\beta(\dot{\theta} - \dot{\psi}) = 2\alpha U \quad (6)$$

$$E_{12} \ddot{\theta} + E_{22} \ddot{\psi} - 2\beta(\dot{\theta} - \dot{\psi}) - Mgl\psi = -2\alpha U \quad (7)$$

avec

$$\alpha = \frac{k_t}{R_m} \text{ et } \beta = \frac{k_t k_b}{R_m} + f_m$$

$$E_{11} = (2m + M)R^2 + 2(J_w + J_m)$$

$$E_{12} = MRl - 2J_m$$

$$E_{22} = Ml^2 + J_\psi + 2J_m$$

2.2 Modèle sous forme de représentation d'état

4. Écrire les équations (6) et (7) sous la forme matricielle suivante en précisant l'expression des matrices E , F , G et H . On rappelle que U est la tension commune des moteurs, soit un scalaire.

$$E \begin{bmatrix} \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} + F \begin{bmatrix} \dot{\theta} \\ \dot{\psi} \end{bmatrix} + G \begin{bmatrix} \theta \\ \psi \end{bmatrix} = HU \quad (8)$$

5. Donner alors l'expression des matrices A_1 et B_1 du modèle d'état $\dot{X} = A_1 X + B_1 U$ du Segway (figure 3) en fonction de E , F , G , H , la matrice identité et la matrice nulle.

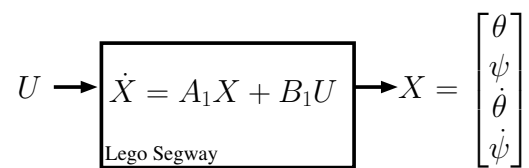


FIGURE 3 – Représentation d'état du système

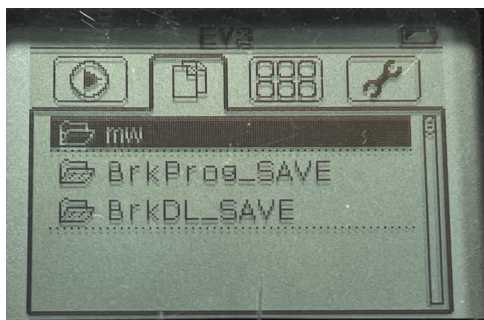
Consignes pour le TP :

- Allumer le PC sous windows. Utiliser le compte local grXY@psi avec X votre numéro de groupe et Y le demi-groupe (a pour le premier, b pour le second).
- Créer un répertoire sur le bureau portant vos noms. Tous vos fichiers Matlab devront être installés ou créés dans celui-ci.
- Répondre de manière concise et précise aux questions dans le fichier Matlab .m. Le cours 1A est fourni avec le reste des fichiers et peut quelquefois être utile.

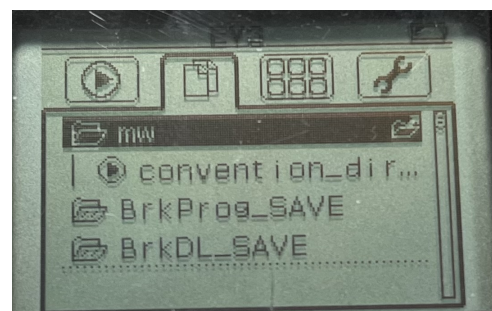
3 TP, partie 0 : Construction du Lego Segway

3.1 Assemblage

1. Dans la page Moodle du module Automatisme et Robotique vous trouverez les fichiers du TP dont le plan de montage du lego nommé Balanc3r. Assemblez le Lego.
2. Une fois assemblé, **mettre la batterie du Lego EV3 en charge avec son chargeur secteur 9V**. Allumer la brique (bouton central-validation), tester le bon fonctionnement des codeurs incrémentaux, du gyroscope, et des moteurs avec les utilitaires Port view et Motor control disponibles dans les menus.
3. Libérer la mémoire disponible en effaçant les programmes installés sur la brique :



(a) Sélectionner le répertoire mw et valider une première fois



(b) Valider une seconde fois



(c) L'icône poubelle apparaît, valider



(d) Valider l'effacement du contenu du répertoire

FIGURE 4 – Procédure d'effacement des programmes

4. Éteindre la brique avec le bouton "retour-annulation" situé en haut à gauche (juste sous l'écran LCD). Laisser en charge.

4 TP, partie 1 : Du modèle à temps continu vers le temps discret

4.1 Analyse du modèle continu

1. Copier dans votre répertoire sur le bureau l'archive du TP disponible dans Moodle. Ouvrir le fichier model.m qu'elle contient. Complétez les paramètres physiques manquants : une balance est notamment à votre disposition.

2. Créer ensuite le modèle d'état du système :
 - Définir les matrices A_1 et B_1 du modèle d'état. Pour cela, il est recommandé de définir d'abord les matrices E, F, G et H en accord avec votre préparation.
 - Créer un système sous forme d'état (state-space) avec la commande `ss`, où l'on choisira pour le moment C_1 de manière à avoir pour sortie du système tous les états.

Note : on peut associer des noms textuels aux états, entrées, sorties du système via les propriétés du modèle `ss` (voir commentaires du fichier `.m`) pour mieux s'y retrouver.
3. Quels sont les états d'équilibre de ce système¹ ? Répondre dans le fichier `.m`.
4. Les équilibres sont-ils stables² ? Justifier.
 - (a) en décrivant une expérience que vous pouvez réaliser avec le Lego Segway devant vous qui montre que l'équilibre est instable ;
 - (b) en utilisant la fonction `pzmap` pour une justification mathématique. Quelles sont les valeurs des pôles retournées par `pzmap` (`help pzmap`) ?

Autovalidation : On donne ici les valeurs approximatives des pôles obtenus (-370, -7, 0, 7).

4.2 Modèle à temps échantillonné du Segway

Le système à temps continu

$$\dot{X}(t) = A_1 X(t) + B_1 U(t) \quad (9)$$

$$y(t) = C_1 X(t) + D_1 U(t) \quad (10)$$

a pour forme à temps discret avec une période d'échantillonnage T_s (en seconde!) :

$$X(k+1) = \Phi(T_s) X(k) + \Gamma(T_s) U(k) \quad (11)$$

$$y(k) = C_1 X(k) + D_1 U(k) \quad (12)$$

où $X(k) = X(t = k T_s)$ et $\{\Phi, \Gamma\}$ sont les matrices d'état et d'entrée du système à temps discret. Les coefficients de ces 2 matrices dépendent de la période d'échantillonnage choisie. Dans le cas d'une entrée $U(k)$ mise à jour à chaque pas d'échantillonnage et dont la valeur est bloquée entre 2 pas, on a la formule $\Phi(T_s) = e^{A_1 T_s}$.

1. Utiliser la fonction `c2d(SYSC, TS, METHOD)` pour obtenir le modèle échantillonné à la période $T_s = 4$ ms du Segway en choisissant la méthode adéquate. Pour choisir la méthode adéquate (`doc c2d`), on sera conscient du fait que le convertisseur numérique-analogique des moteurs dans la brique conserve la tension de sortie demandée (comme tout CNA) tant qu'il ne reçoit pas une nouvelle consigne de tension.
2. Vérifier que la matrice d'état du modèle discret obtenue est en accord avec la formule donnée plus haut pour Φ (fonction `Matlab expm`). On accède aux matrices d'un modèle `my_model` sous forme `ss` avec `my_model.a` pour la matrice d'état A par exemple.
3. Si on analyse les pôles du système discret (avec `eig` ou `pzmap`), on constate qu'il existe une correspondance avec les pôles du système continu donnée par la relation :

$$z_p = e^{s_p T_s}$$

où z_p représente le pôle discret et s_p le pôle continu.

La limite entre le demi-plan stable et le demi-plan des pôles instables en temps continu est l'axe des imaginaires purs dans le plan complexe. D'après la relation précédente et l'équation paramétrique de l'axe des imaginaires purs ($s = jw$ avec $w : -\infty \rightarrow +\infty$), quelle est la limite de stabilité dans le plan complexe en temps discret ?

4. Avant de se lancer dans une correction/stabilisation par retour d'état, ce système discret qu'est le Segway à temps échantillonné est-il commandable (fonction `rank`) ?
- Le test de commandabilité est similaire en discret au cas continu en substituant les matrices $\{\Phi, \Gamma\}$ aux matrices $\{A, B\}$. Pourquoi est-il important de vérifier a priori cette propriété du système ?

1. on considère X stationnaire et $U = 0$ dans la représentation d'état pour les calculer.
 2. leur stabilité dépend de la stabilité des pôles.

5 TP, partie 2 : Stabilisation par retour d'état

Pour rappel, un retour d'état statique (figure 5) consiste à imposer une commande stabilisatrice au système de la forme :

$$U = -K x(k) \quad (13)$$

alors l'expression en boucle fermée du système est :

$$X(k+1) = \Phi X(k) + \Gamma U(k) \quad (14)$$

$$= \Phi X(k) - \Gamma K X(k) \quad (15)$$

$$X(k+1) = (\Phi - \Gamma K) X(k) \quad (16)$$

Il suffit alors pour stabiliser le système de choisir le vecteur de gain K tel que la matrice d'état $(\Phi - \Gamma K)$ a des valeurs propres (pôles z_p) stables.

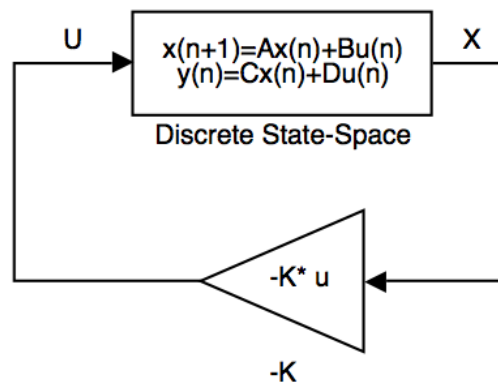


FIGURE 5 – Schéma bloc de principe du retour d'état : $u = -Kx$

1. Dans le cas où K vérifie la condition ci-dessus pour le système corrigé (i.e $|z_{pi}| < 1$), vers quelle valeur $X_{eq}(k)$ converge $X(k)$ quand $k \rightarrow \infty$? La valeur finale X_{eq} est-elle effectivement un équilibre du Segway?

On peut s'inspirer du cas diagonal simple ci-dessous avec $X_{eq}(k) = [x_1, x_2]^T$ pour le raisonnement :

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \end{bmatrix} = \begin{bmatrix} z_{p1} & 0 \\ 0 & z_{p2} \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} \quad (17)$$

Pour déterminer K dans le retour d'état, il faut choisir les pôles souhaités du système corrigé : c'est un placement de pôles.

On propose ici pour le choix des pôles du système bouclé :

- de conserver les 2 pôles stables et rapides présents dans le système discret non corrigé²;
- et de choisir un pôle dominant réel ≥ 0 de multiplicité 2 pour les pôles restants : l'approche simple et classique.

2. Quel pôle appelle-t-on "dominant" parmi un ensemble de pôles : le plus lent ou le plus rapide? Le dominant est celui que l'on ne peut pas négliger pour rendre compte de la dynamique du système. Avec la relation précédente $z_p = e^{s_p T_s}$ et pour se faire une idée des constantes de temps en jeu avec une période d'échantillonnage $T_s = 4\text{ms}$, quelles constantes de temps sont associées aux 2 pôles stables du Segway à temps discret? En discret, un pôle lent est-il proche de 0 ou du cercle unité?
3. Utiliser alors la formule d'Ackermann (voir aide de la fonction `acker`) pour calculer le gain K du retour d'état permettant d'imposer les pôles souhaités au système bouclé. Pour le choix du pôle dominant multiple, on prendra soin à trouver un compromis entre un pôle trop rapide (qui demande une tension

de commande trop grande³) et un pôle trop lent (l'état du système s'éloigne trop du point d'équilibre et l'approximation modèle linéaire n'est plus valide).

Pour chercher une valeur optimale de ce pôle, il vous faut implémenter et simuler le retour d'état Fig. 5 via Simulink avec un gain et un bloc state-space discret de période T_s . Pour des vitesses nulles et une position angulaire ψ initiale limite de 10 degrés (à définir dans les bonnes unités dans le vecteur d'état initial du bloc state-space), la commande doit rester compatible avec la charge maximale de la batterie et ne doit donc pas dépasser les 10 volts.

Vérifier sur les courbes que le système est bien stabilisé à l'équilibre.

Rappel : On peut utiliser toute variable Matlab définie dans l'espace de travail comme paramètre d'un bloc Simulink.

[Point de validation : Faire valider par un encadrant.]

2. Une alternative consiste à conserver uniquement le pôle très rapide et choisir un pôle triple pour les pôles restants.

3. En effet, rien en simulation n'empêche de choisir les pôles très rapides pour une réponse instantanée du système (1 femto seconde) mais ce temps de réponse est alors associé à une commande du système physiquement irréaliste (10^{13} volts).

6 TP, partie 3 : Implémentation du retour d'état pour le Segway Lego EV3

Dans cette section, on se propose d'abord d'implémenter le retour d'état sous Simulink autour d'un simulateur du Lego Ev3.

Une fois le schéma validé, on utilisera la capacité de Simulink à transformer un schéma blocs en code C pour programmer le Lego.

6.1 Analyse du simulateur Ev3

Le Lego EV3 (modèle linéaire) est simulé dans le sous-système du fichier `simulation.slx` avec comme sur la maquette :

- en entrée, une tension en pour cent de la batterie U_{pwm} (Pulse Width Modulation (pwm) : signal de rapport cyclique variable entre -100 et 100)
- en sortie, la mesure du gyroscope (deg/s) et les angles moteurs en (deg) tels que délivrés par les capteurs du lego et comme préciser dans la section 1.2.

1. Ouvrir le sous-système présent dans la simulation. Au centre, on a le modèle d'état continu de la dynamique du système. Vérifier dans ce bloc que les paramètres correspondent bien aux variables que vous avez définies. Noter la condition initiale non nulle, initialisée à $[0 \ \psi_0 \ 0 \ 0]^T$.

On trouve en entrée un gain $\frac{1}{K_{pwm}}$, avec K_{pwm} le gain de conversion d'une tension (volt) en un rapport cyclique du signal PWM ($-100\% < \dots < 100\%$). En sortie, un bloc fonction (que l'on peut ouvrir) modélise les mesures fournies par les capteurs Lego avec l'unité d'angle propre aux capteurs Lego : le degré. Ces mesures sont alors échantillonnées à T_s par le bloc suivant.

Quelle propriété des capteurs modélisent les blocs `round` présents (voir section 1.2)?

6.2 Implémentation du retour d'état

Le simulateur, comme la maquette du Segway Ev3, ne donne pas accès à l'ensemble des états du vecteur d'état X , dont on a besoin pour la commande via le retour d'état.

Il faut donc commencer par reconstruire le vecteur d'état $\begin{bmatrix} \theta & \psi & \dot{\theta} & \dot{\psi} \end{bmatrix}^T$ dans les bonnes unités, à partir des données fournies par les 2 seuls capteurs disponibles (figure 6).

1. En dehors du bloc EV3 (Simulateur), commencer par reconstruire le signal $\dot{\psi}$ dans les unités attendues en connectant un bloc `Gain` à la sortie `gyro` du simulateur.
2. Pour reconstruire l'angle ψ , on n'utilisera pas directement la mesure de $\dot{\psi}$ car celle-ci peut contenir un faible biais constant (offset) dû à la technologie du capteur. L'intégration directe du signal avec un biais constant ne peut que diverger. On se propose donc d'estimer ce biais par un filtrage et de le soustraire à la mesure de $\dot{\psi}$ issue du gyroscope pour estimer une valeur non biaisée (voir figure 6).

Le filtre retenu pour estimer le biais est un filtre moyennneur à fenêtre glissante et facteur d'oubli exponentiel ("exponential smoothing" en anglais) qui se traduit par l'équation aux différences suivante (équation utilisée quand il faut implémenter le filtre sur un microprocesseur) :

$$\text{biais}(k) = \gamma \text{biais}(k-1) + (1-\gamma)u(k) \quad (18)$$

avec $\text{biais}(k)$ le biais à estimer, $u(k)$ le signal en entrée et un facteur d'oubli $0 < \gamma < 1$.

Donner l'expression de la fonction de transfert discrète en z ou z^{-1} correspondant à ce filtre.

Rappel : $\mathbf{Z}\{y(k-1)\} = z^{-1}Y(z)$ avec $Y(z) = \mathbf{Z}\{y(k)\}$.

3. Le facteur d'oubli choisi est $\gamma = 0.999$. Afin de valider ce choix et votre résultat précédent, créer la fonction de transfert échantillonnée à T_s ms pour ce filtre (`help tf`). Tracer son diagramme de Bode (fonction `bode`). Est-ce un filtre passe-bas ou passe-haut? Quels sont sa fréquence de coupure et le gain en basse fréquence? Est-ce un filtre adéquat pour identifier un biais constant?
4. Avec les blocs Simulink de base (gain, sommateur, mux, demux,...) et blocs de fonctions de transfert discrètes³, implémentez alors l'estimation de l'état ψ .

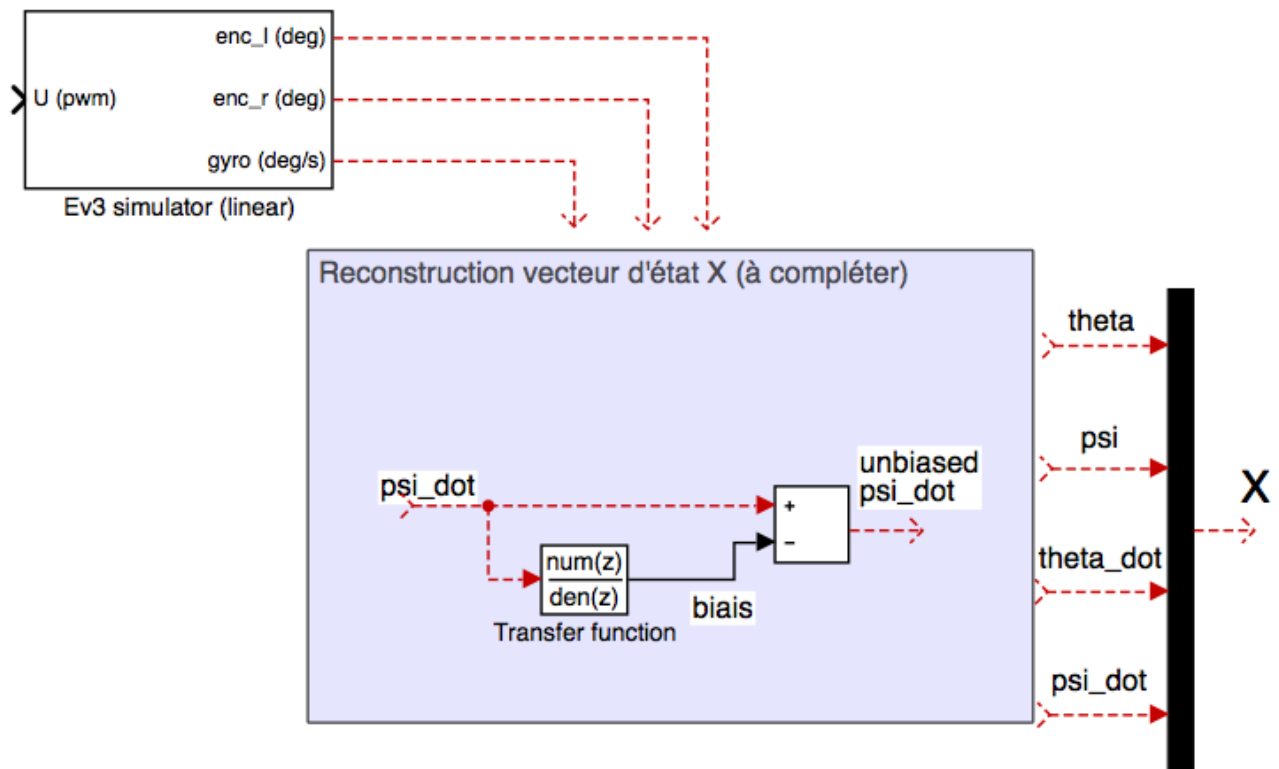


FIGURE 6 – Schéma bloc de la reconstruction de l'état X et estimation d'une vitesse angulaire $\dot{\psi}$ non biaisée pour l'intégration

- Implémenter de même le calcul de la position angulaire moyenne θ du Segway et sa dérivée $\dot{\theta}$ à partir des signaux issus du codeur droit et gauche du simulateur du Segway Ev3 (voir détail section 1.2).
- Ayant désormais reconstruit à partir des capteurs disponibles le vecteur d'état X (figure 6), stabiliser le système en ajoutant le gain du retour d'état. Valider par la simulation et l'observation de l'évolution des états.

En cas de problème, vérifier que la commande en entrée du simulateur Ev3 est bien dans la bonne unité (Quelle est la grandeur en sortie du gain du retour d'état, figure 5 ? Regarder également à l'intérieur du bloc simulateur).

Rappel : Le vecteur d'état initial dans le simulateur est non nul, sinon il ne se passe rien, car on est déjà à l'équilibre en 0.

[Point de validation : Faire valider par un encadrant]

6.3 Premiers pas avec la programmation via Simulink du Lego Ev3

Dans Simulink, créer une nouvelle simulation vierge. Aller dans la bibliothèque pour trouver les divers blocs permettant de reproduire la simulation de la figure 7.

Cette simulation simple envoie une consigne constante de 10% au moteur connecté au port A, lit la valeur entière retournée par les capteurs (degrés), convertit cette valeur en double et affiche le résultat sur une ligne de l'écran LCD. Les blocs *conversion* (disponibles dans la section Signal Attributes de la bibliothèque) sont nécessaires, car les autres blocs Simulink classiques travaillent uniquement en précision double. Penser à :

- configurer le port de chaque bloc Ev3 et indiquer T_s comme fréquence d'échantillonnage dans les propriétés des capteurs ;
- choisir une ligne d'affichage différente pour le codeur et le gyroscope.

3. On s'interdit d'utiliser des blocs contenant des fonctions de transfert continues (en s), car :

- un calculateur numérique travaille à temps échantillonné et donc l'état sera reconstruit à temps échantillonné.
- toutes les fonctions de transfert doivent être échantillonnées et de période commune T_s afin que Simulink puisse générer le code C correspondant à la simulation.

1. Allumer et connecter la brique Ev3 au PC avec le câble USB fourni. Pour compiler et exécuter ce schéma-blocs sur la brique, il vous faut la toute première fois aller dans l'onglet `Modeling` de `simulink`, puis sélectionner `Model Settings`, `Model configuration` et dans le panneau de configuration qui s'ouvre `Hardware implementation`. Choisir `LEGO EV3` pour la plateforme `Hardware`. (A priori les paramètres par défaut sont corrects. Si besoin `build`, `load` and `run` pour `build options` dans `hardware ressource`, ainsi que l'usb pour la `connection host to hardware connection`.) Appliquer les changements et fermer le panneau.
2. Cliquez sur l'icône "Build, deploy & start" dans le bandeau `Simulink`, onglet `Modeling`. Le programme compile puis se lance automatiquement sur la brique après la fin de la compilation.
 Note 1 : Si on modifie la simulation, un clic sur l'icône suffit pour lancer la nouvelle simulation.
 Note 2 : Après une première exécution, on peut lancer le programme sans être connecté au PC en allant dans l'explorateur de fichiers de la brique, répertoire `mw` et sélectionner le nom de la simulation.
3. Grâce à cette simulation, noter le sens positif défini par `Simulink` pour la rotation des moteurs, les codeurs et le capteur gyroscopique. Correspondent-ils au choix fait lors de la modélisation figure 2 ? Sinon, mettre des gains -1 sur les entrées et sorties qui le nécessitent afin d'avoir les mêmes conventions de signe que dans la modélisation (et donc le simulateur Ev3 précédent).

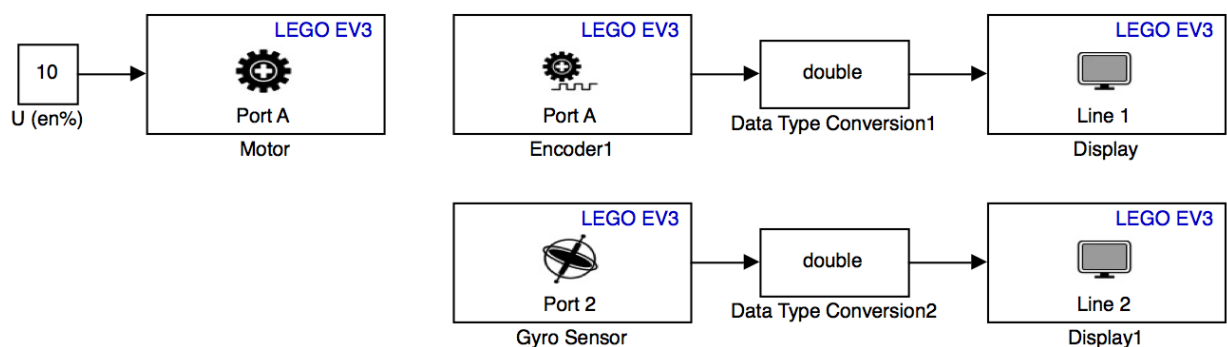


FIGURE 7 – Schéma bloc pour tester les conventions de sens de rotation positif des blocs Ev3

6.4 Implémentation du retour d'état sur le matériel Lego Ev3

L'heure de tester sur la maquette est venue. Le segway va-t-il rester en équilibre ? Si votre retour d'état est effectif avec le simulateur du lego (`simulation.slx`), il suffit alors de remplacer le contenu du bloc `Ev3 (simulateur)` par les blocs `Ev3` adéquats de connexions aux moteurs et capteurs de la brique Lego Ev3. Ensuite, `Simulink` se chargera de générer le code C correspondant à la simulation et de l'uploader sur la brique.

1. Dupliquer votre fichier `simulation.slx` sous un autre nom. Dans cette copie, renommer le sous-système `Lego EV3 (Simulateur)` en `Lego Ev3 (hardware)`. Ouvrir le sous-système et effacer tous les blocs à l'exception du port d'entrée `U` et des 3 ports de sorties `enc_l`, `enc_r` et `gyro`.
2. Connecter alors les ports d'entrées et sorties aux blocs `Ev3` motor, `Encoder` et `Gyro Sensor` correspondant afin de remplacer le simulateur par le système réel. Comme dans votre simulation précédente, penser à :
 - ajouter les blocs `conversions` en double en sortie des capteurs ;
 - ajouter un gain -1 si le signe du capteur n'est pas le même que celui du modèle ;
 - dans les propriétés de chaque bloc, définir le port de connexion et T_s comme fréquence d'échantillonnage des capteurs.
3. Préparer le déploiement sur le Lego Ev3 comme précédemment via l'onglet `Modeling` dans le bandeau `Simulink`. Configurer et déployer sur le Lego.
 Tester :
 - sélectionner le programme dans le menu ;

- mettre le segway à la verticale : chercher manuellement le point d'équilibre vertical du Lego (il faut qu'au démarrage, $\psi = 0$ car on obtient ψ par intégration en partant de 0);
- tenir délicatement le Segway et appuyer sur le bouton central pour lancer le programme;
- lâcher immédiatement.

Si tout a bien été fait, il reste en équilibre!

[Point de validation : Faire valider par un encadrant]

7 Asservissement en poursuite du Segway Ev3

Souvent, en plus de stabiliser le système, on veut pouvoir imposer son régime permanent. En particulier asservir la sortie du système Y à une consigne de référence Y_{ref} . On introduit donc une référence dans le schéma de commande et un gain de préfiltre $\bar{N}$ qui assure un gain statique de 1 entre Y et Y_{ref} en régime permanent (figure 8).

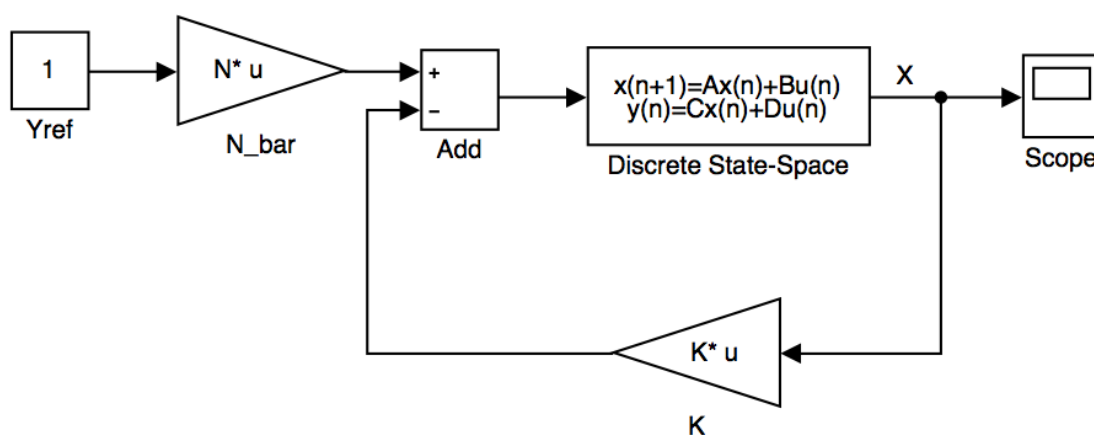


FIGURE 8 – Structure complète d'une commande par retour d'état

1. Les seuls régimes permanents correspondent à un état d'équilibre. Quelle variable du vecteur d'état est-il possible d'imposer à une référence dans le cas du Lego Segway?
2. Le schéma de régulation figure 8 est décrit par la représentation d'état suivante :

$$X(k+1) = (\Phi - \Gamma K)X(k) + \Gamma \bar{N} Y_{ref} \quad (19)$$

$$Y = C X(k) \quad (20)$$

Une fois le régime permanent atteint, par définition, quelle relation lie $X(k+1)$ et $X(k)$?
Montrer alors qu'en régime permanent et avec les équations précédentes, $Y = Y_{ref}$ impose que $\bar{N} = [C(I + \Gamma K - \Phi)^{-1} \Gamma]^{-1}$ dans le cas discret.

3. Calculer la valeur du gain $\bar{N}$ en choisissant C en fonction de la variable de sortie à réguler dans le système ci-dessus. Compléter `Simulation.slx` contenant le simulateur du lego Ev3 avec la structure complète du retour d'état. Vérifier alors qu'en régime permanent la sortie est égale à la consigne.

7.1 Détection d'obstacle et asservissement en poursuite du Lego Segway Ev3

1. Imposer θ , soit la position au sol $x = r\theta$, est souvent moins intéressant qu'imposer une vitesse. De plus, il n'est pas possible de demander une valeur de θ_{ref} très différente de la valeur courante, θ car cela se traduit par un transitoire de commande très élevé.
On préfère donc ne pas donner des échelons de référence, mais une rampe de position, c.-à-d. piloter en vitesse : remplacer la consigne constante de position par une consigne croissante de position en utilisant un intégrateur discret et une consigne de vitesse angulaire $\dot{\theta}_{ref}$. Tester en simulation.

2. Tester ensuite sur la maquette Lego Segway en imposant une vitesse constante (pas trop rapide).
3. On souhaite asservir le Segway tel qu'il avance à vitesse constante en l'absence d'obstacle et recule à vitesse constante en présence d'un obstacle.
 Modifier le schéma-bloc en ajoutant le bloc capteur à ultrasons dans le sous-système lego Ev3. Ce capteur renvoie une distance en cm. Utiliser alors un bloc switch (figure 9) qui permet de basculer d'une consigne à une autre en fonction d'une condition sur un signal externe (la mesure du capteur à ultrasons).

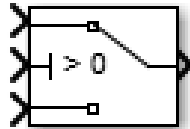


FIGURE 9 – Block switch

8 TP, partie finale : Démontage du Lego Segway

Démonter le robot. Mettre dans le fond de la boîte les éléments de la figure 10!!
 Laisser votre session Matlab ouverte.

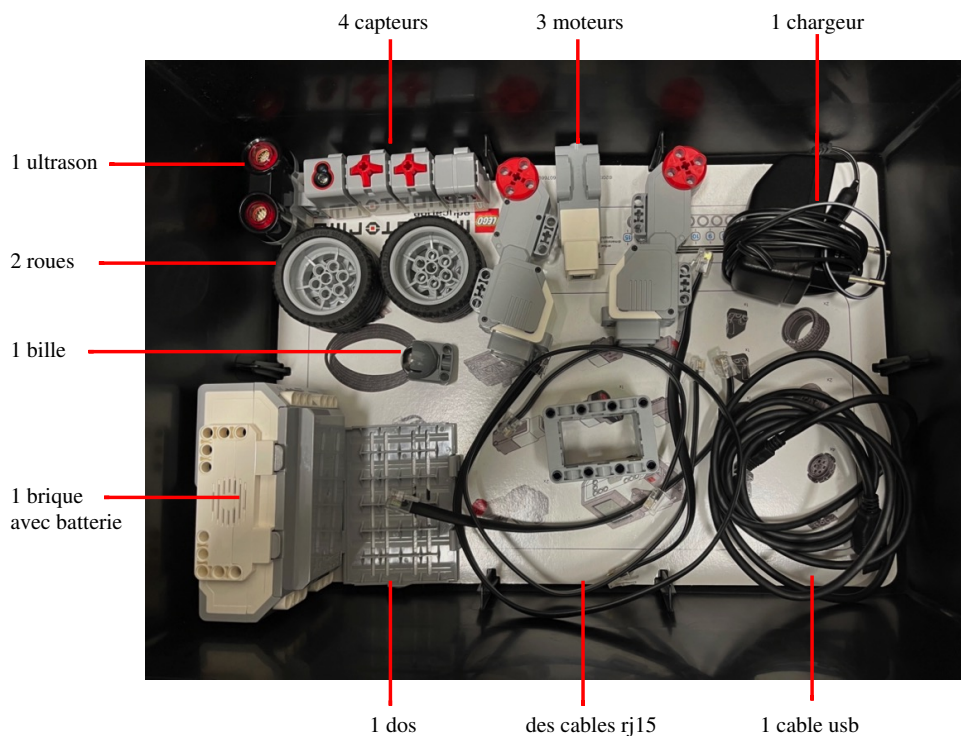


FIGURE 10 – Contenu attendu au fond de la boîte